Steiner Tree and Traveling Salesman Problem

Master in Digital Technology

Student ID: 603927

TU Clausthal

Motivation

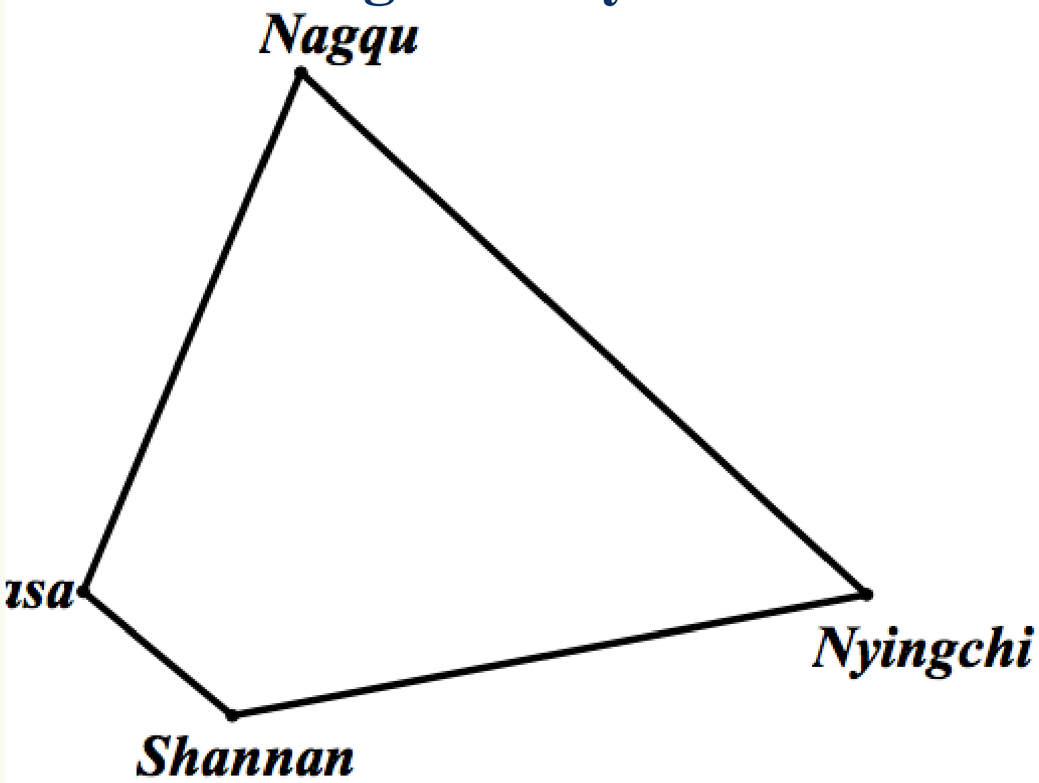
Why Study These Problems?

- Many real-world problems require connecting locations at minimum cost
  - Examples:
    - Network design (fiber, internet, roads)
    - VLSI chip wiring
    - Biological phylogenetic trees
    - Logistics and route planning
- Two fundamental questions arise:
  - How to connect points cheaply? (Steiner Tree)
  - How to visit all points efficiently? (TSP)
- Exact solutions are computationally infeasible
- We therefore study approximation algorithms



City Connections Example

Cities connecting directly



Cities connected with junction

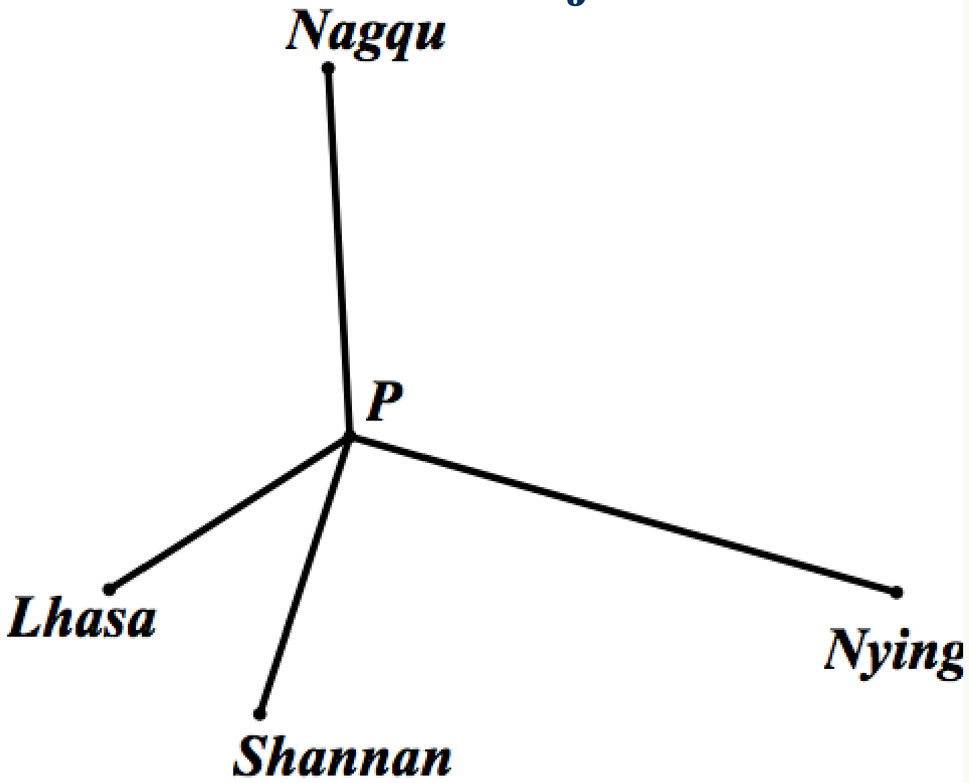


Figure 2 Two possible arrangements for building Roads

The Steiner Tree Problem

Connecting terminals with minimum cost

Block 1: Problem 3.1(Steiner Tree)

Concept

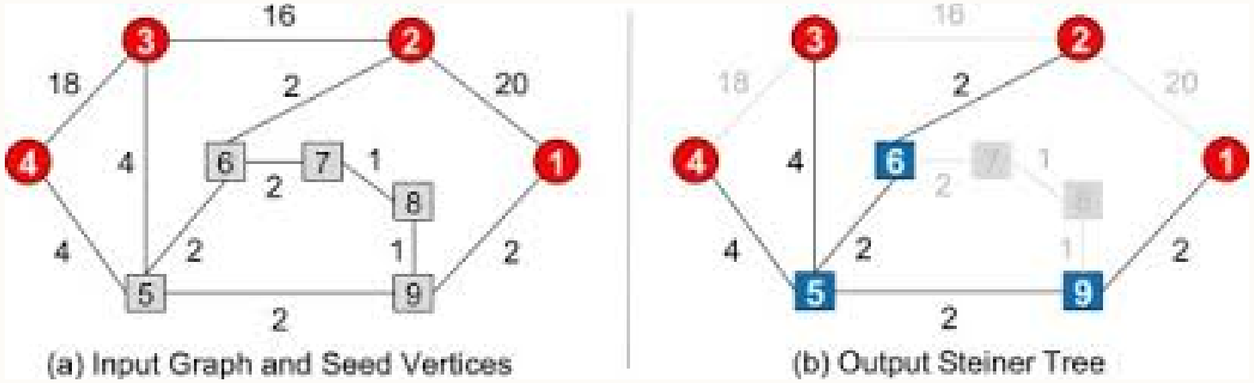
Given:

- An undirected graph
  - $G = (V , E)$
- Each edge has a non-negative cost
- Vertices are split into two groups:
  - Required vertices (terminals)
  - Steiner vertices (optional)

Objective:

- Find a minimum-cost tree that:
  - Contains all required vertices
  - May contain any subset of Steiner vertices

Diagram: Showing : input graph and Steiner Tree



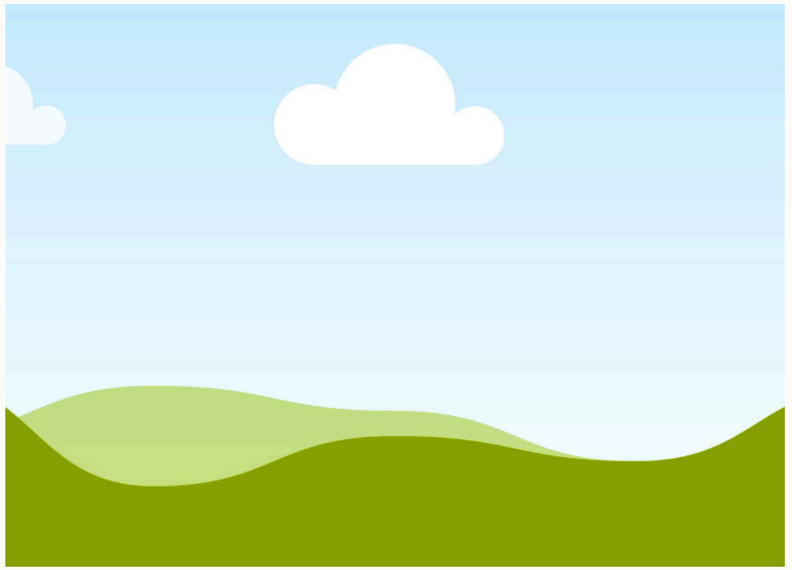
Block 1: Metric Steiner Tree

Concept

Given:

- We focus on the Metric Case.
- Costs are non-negative.
- The graph is complete.
- Crucial Rule: Triangle Inequality holds.

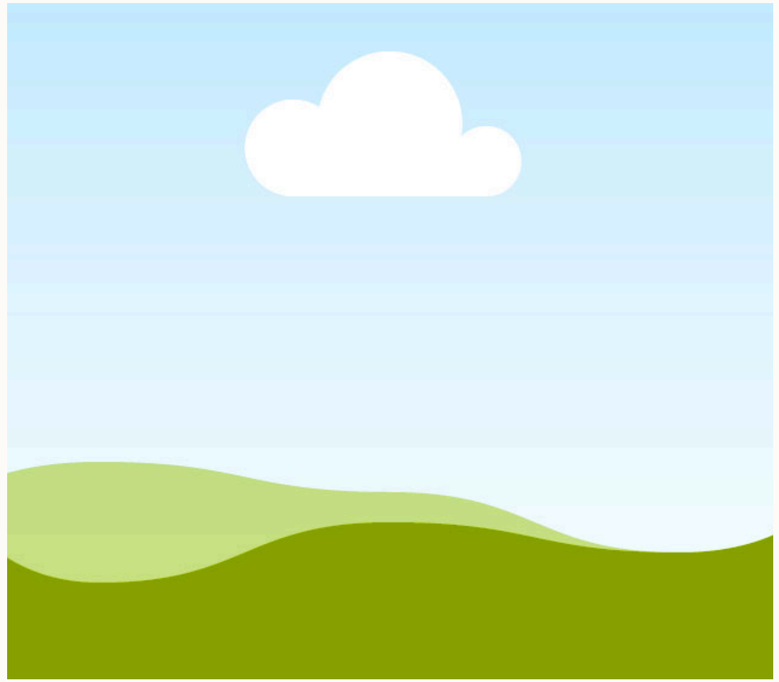
Simple Triangle u,v,w show direct path u to v is shorter than going u - w to v.



Block 1: Theorem 3.2: The Reduction

- There is an approximation-factor preserving reduction from Steiner Tree to Metric Steiner Tree.
- Approximation-Factor Preserving: The "closeness" to the perfect answer remains the same in both versions.
- If we solve the "Metric" version, we have effectively solved the "General" version without losing any quality in our answer.

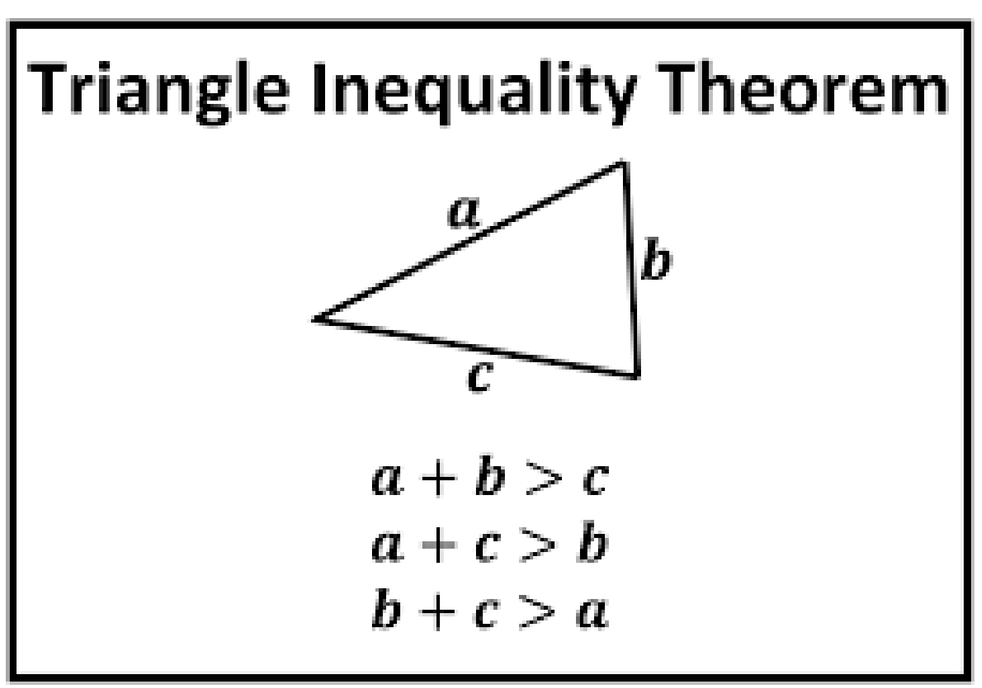
Visual: An arrow showing: [General Problem] \to [Reduction] \to [Metric Problem]



Block 1: Traingle Inequality

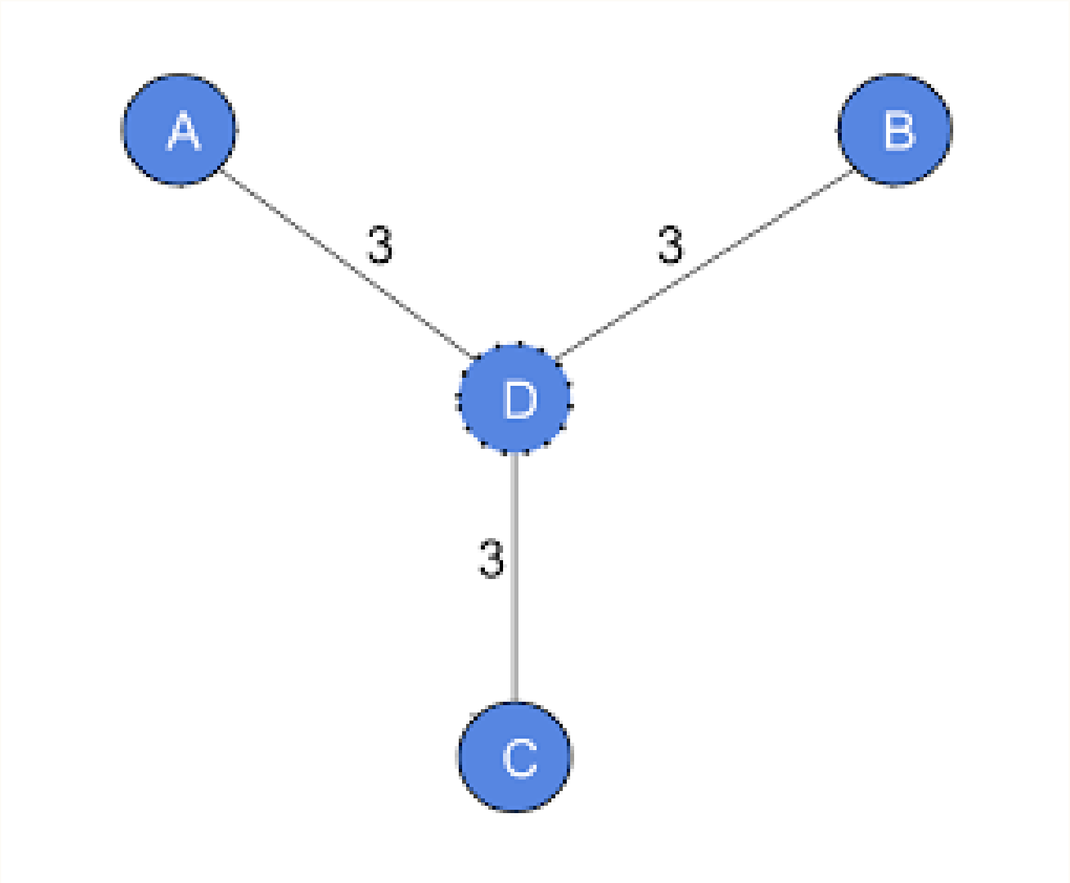
- The core of the problem lies in how edge costs behave
- We focus on instances where:
  - The graph is complete
  - Edge costs are non-negative
  - Triangle inequality holds:
    - cost(u,v) ≤ cost(u,w) + cost(w,v)
- Triangle inequality ensures distances behave realistically
- In a metric world, the shortest distance between two points is a direct line.

Trainagle Inequality Theorem

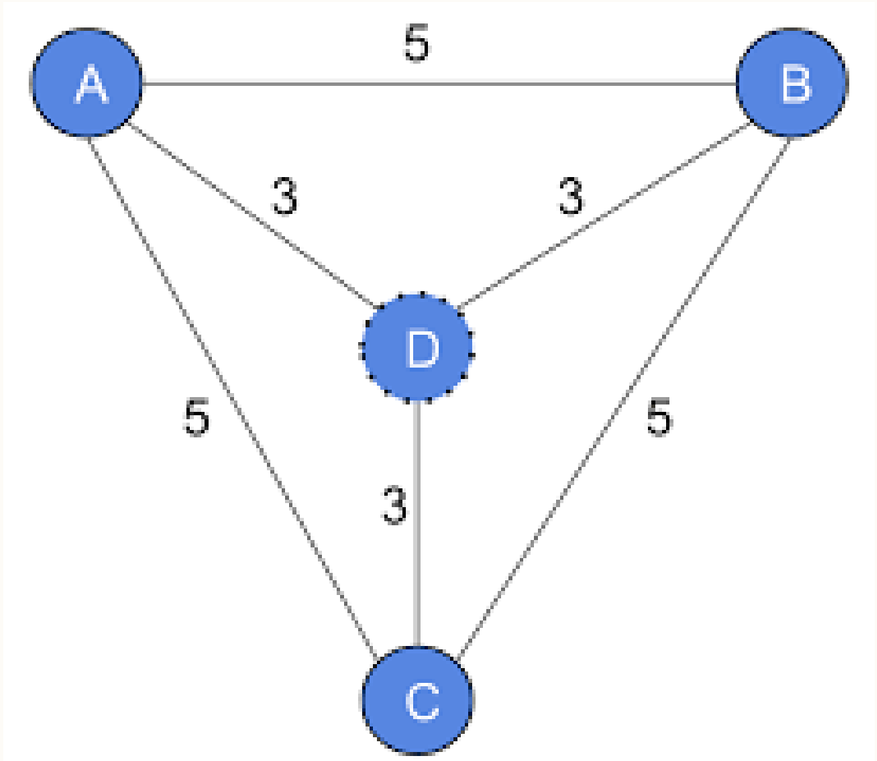


Steiner Tree and Complete Graph

Steiner Tree



Complete Graph



Block 1: Theorem 3.2 Proof : Part 1: Metric Closure

- Points:
- Take any graph G.
- Construct G' (Metric Closure): A complete graph where the cost of (u,v) is the shortest path distance in G.
- Visual: A graph with a winding path between A and B. In G', replace that winding path with a single direct edge of the same cost.

Block 1: Proof 3.2 : Part 2: Comparing Costs

- Logic: cost(u,v) in G' is ≤ cost(u,v) in G.
- Therefore, the perfect answer in the "Shortcut Map" (G') is at least as cheap as the perfect answer in the "Real Map" (G).
- OPT(G') ≤ OPT(G).
-
- Visual: A "Price Match" symbol. The metric map always matches or beats the original price.
-

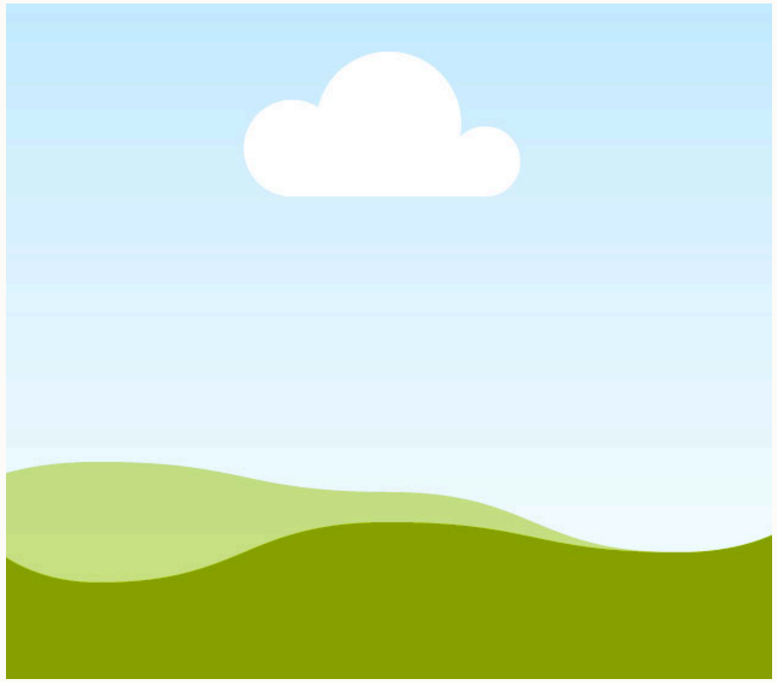
Block 1: Proof 3.2 : Part 3: Mapping

- Logic:
- Take a Steiner Tree T' from the metric map.
- Replace each "Shortcut" with the "Real Path" from the original map.
- If you get loops (cycles), delete extra edges.
- Cost only goes down or stays same!
- Visual: A straight line A-B being replaced by its original zigzag path. Then show a circle being "cut" into a tree.

Block 2: MST-based Algorithm

- Concept:
  - Let R be the set of Required vertices.
  - An MST on R is a feasible solution (it connects everything we need).
- The Comparison:
  - Finding an MST is in P (Fast).
  - Finding a Metric Steiner Tree is NP-hard (Slow).
  - We use the MST as a fast substitute, even if it's slightly more expensive.
- Theorem 3.3: The cost of an MST on R is within 2 · OPT.

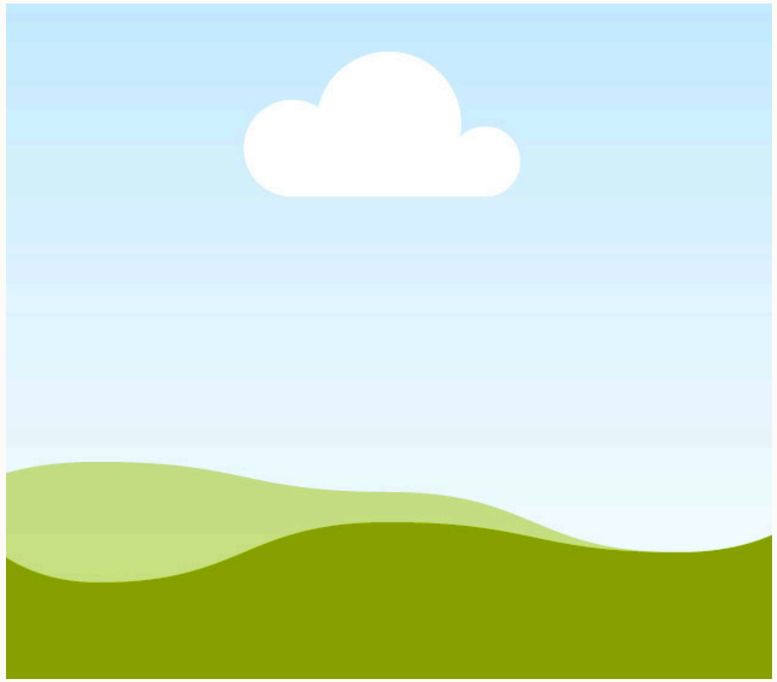
Visual: A graph showing required nodes connected via a simple MST compared to a more complex Steiner tree using extra points.



Block 2: Proof of Theorem 3.3 (Part 1) Doubling and Touring

- Step 1: Double the Edges
  - Take an optimal Steiner tree (Cost = *OPT*).
  - Double every edge (Cost = 2 . *OPT*).
  - This creates an Eulerian graph (every node has even degree).
- Step 2: Find an Euler Tour
  - Use DFS to find a tour that visits every vertex and uses every edge exactly once.
  - Tour Cost = 2 . OPT.

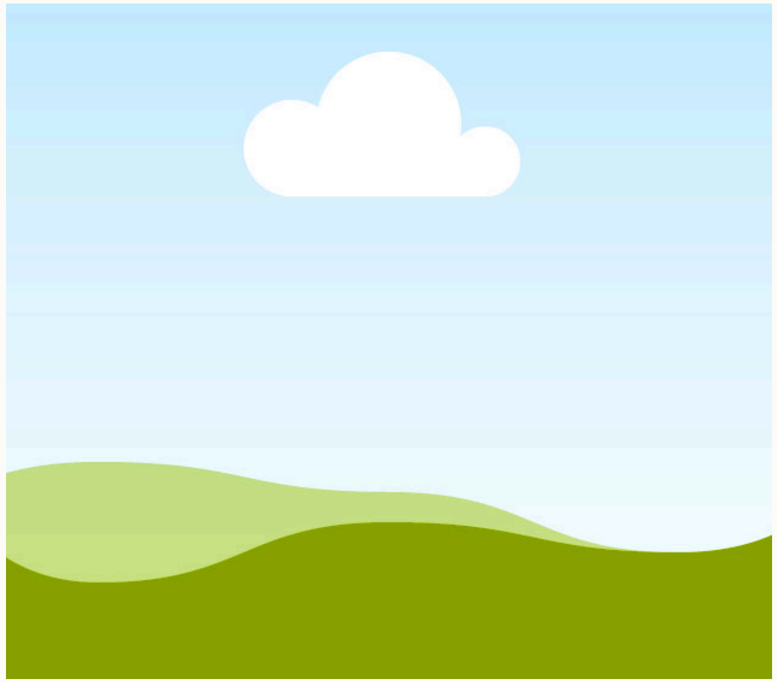
Visual: A tree diagram where each branch is replaced by two parallel lines, and a dotted arrow showing a full "loop" around the tree.



Block 2: Proof of Theorem 3.3 (Part 2) Short-cutting

- Step 3: Short-cut to Hamiltonian Cycle
  - Traverse the Euler tour.
  - If a Steiner vertex or a previously visited vertex appears, skip it and go to the next unvisited required vertex.
- The Logic (Triangle Inequality):
  - Because the graph is metric, skipping points (short-cutting) never increases the cost.
  - We now have a Hamiltonian Cycle on R with cost $\leq 2 \cdot OPT$.
- Final Step:
  - Delete one edge from this cycle.
  - We get a path on R with cost $\leq 2 \cdot OPT$.

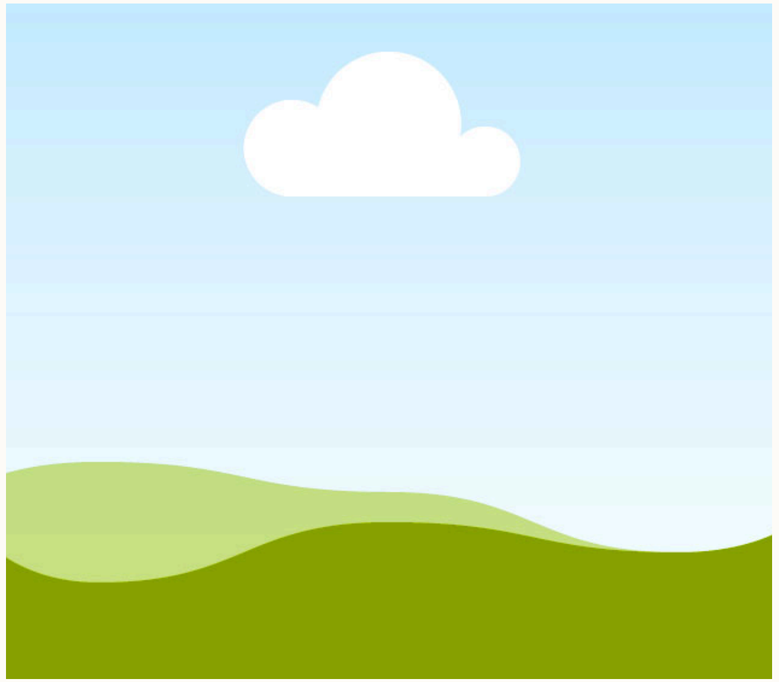
Visual: Short Cut to Hamiltonian Cycle



Block 2: Example

- The Scenario:
  - *n* required vertices on a circle.
  - 1 Steiner vertex in the dead center.
  - Distance from center to any required vertex = 1.
  - Distance between any two required vertices = 2.
- The Math:
  - Optimal (OPT): Connect all n nodes to the center Steiner node. Cost = n.
  - MST on R: Since we can't use the center, we connect nodes along the perimeter. Cost = 2(n-1).
- The Ratio:
  - As *n* gets larger, the ratio 2(n-1)/ n approaches 2.

Visual: A "Star Graph" with n points. Show the center node being used by the Steiner tree but avoided by the MST.

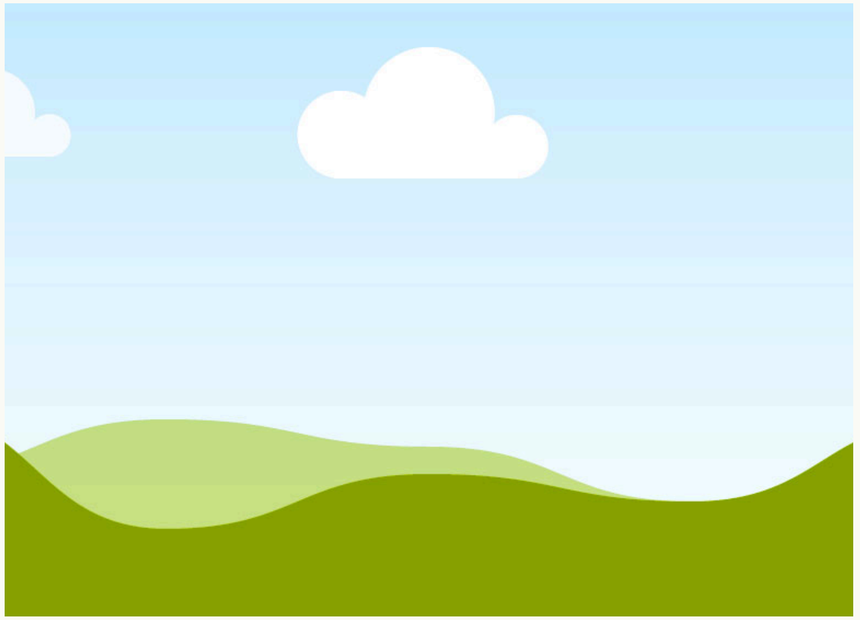


The Traveling Salesman Problem

What is TSP?

- What is TSP?
  - Imagine a delivery driver who needs to visit every city and return home.
- The Goal:
  - Find the absolute shortest path that visits every city exactly once and returns to the start.
- Why it's hard:
  - The number of possible paths grows incredibly fast (Factorial!).
- Visual: A map with cities and a single loop connecting them all efficiently.

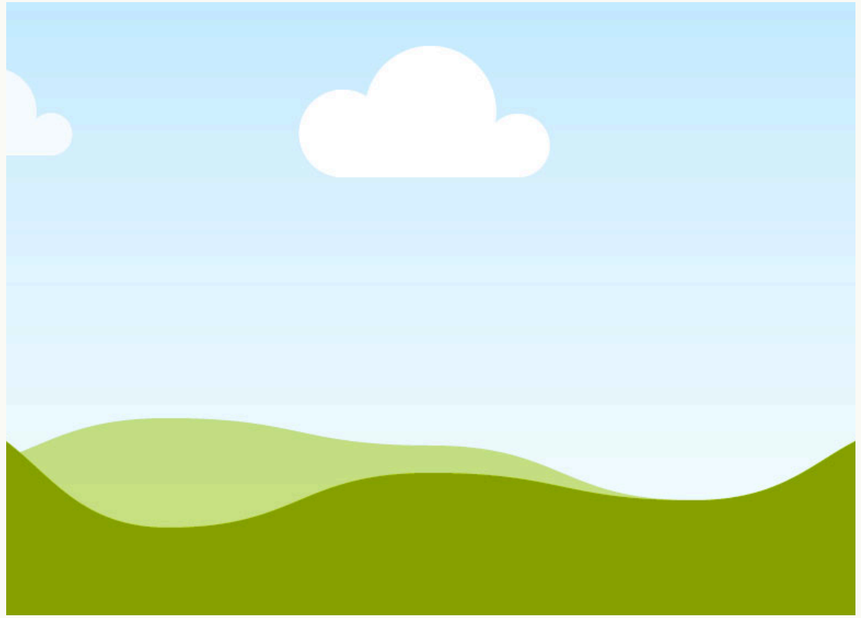
Visual: A map with cities and a single loop connecting them all efficiently.



Problem 3.5 - The Traveling Salesman Problem

- Definition:
  - Given a complete graph with non-negative edge costs.
  - Find a minimum cost cycle visiting every vertex exactly once.
- The Hard Truth:
  - In its full generality, TSP cannot be approximated at all (assuming P /= NP).

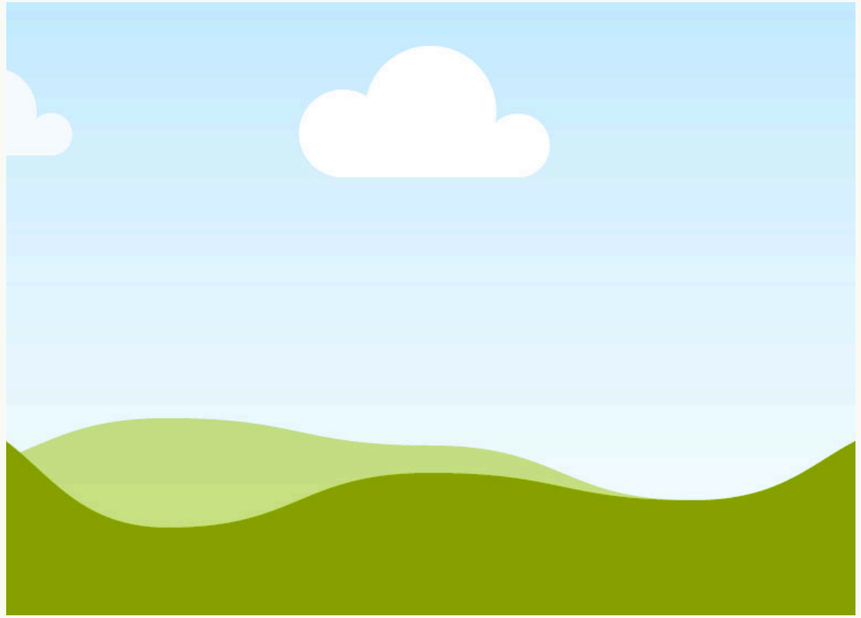
Visual: A graph with 5 vertices. Show one possible tour (1 \to 2 \to 3 \to 4 \to 5 \to 1).



Theorem 3.6 - Hardness of General TSP

- Statement:
  - For any polynomial time computable function α(n), TSP cannot be approximated within a factor of α(n), unless P = NP.
- Simple Explanation:
  - Even if you ask for an answer within a factor of 100 or 1,000,000, it's still impossible to guarantee.

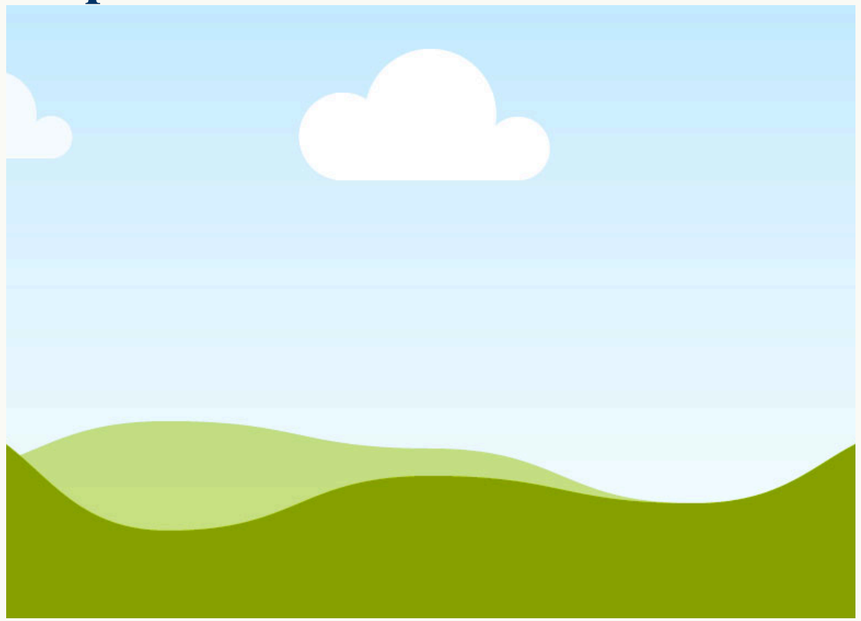
Visual: A "Road Closed" sign on the way to approximating General TSP.



Proof of Theorem 3.6 (Part 1) - The Reduction

- The Goal: Show that approximating TSP is as hard as the Hamiltonian Cycle problem.
- The Setup:
  - Transform graph G into a complete graph G'.
- Assigning Weights:
  - If edge exists in G is to Cost is 1.
  - If edge NOT in G is to Cost is $\alpha(n) \cdot n$ (Huge!).

Visual: Graph being turned into a complete graph where "Missing" edges are thick, expensive red lines.

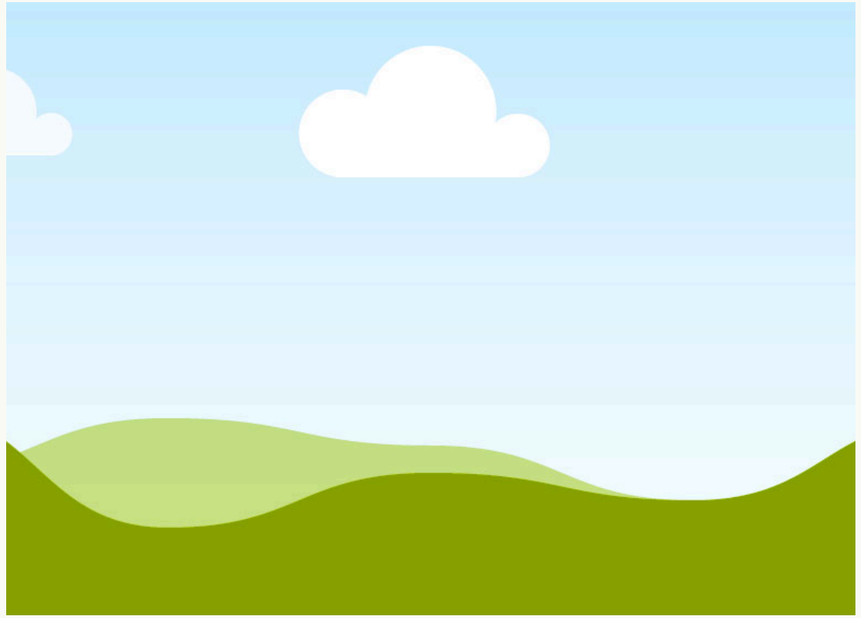


Proof of Theorem 3.6 (Part 2) - The Gap

- Case 1: Hamiltonian Cycle exists
  - Tour uses only "1" edges. Total Cost = n.
- Case 2: No Hamiltonian Cycle exists
  - Tour must use at least one "missing" edge.

    Total Cost > α (n) . n.
- Conclusion: An approximation algorithm would easily see the "Gap" and solve an NP-hard problem.

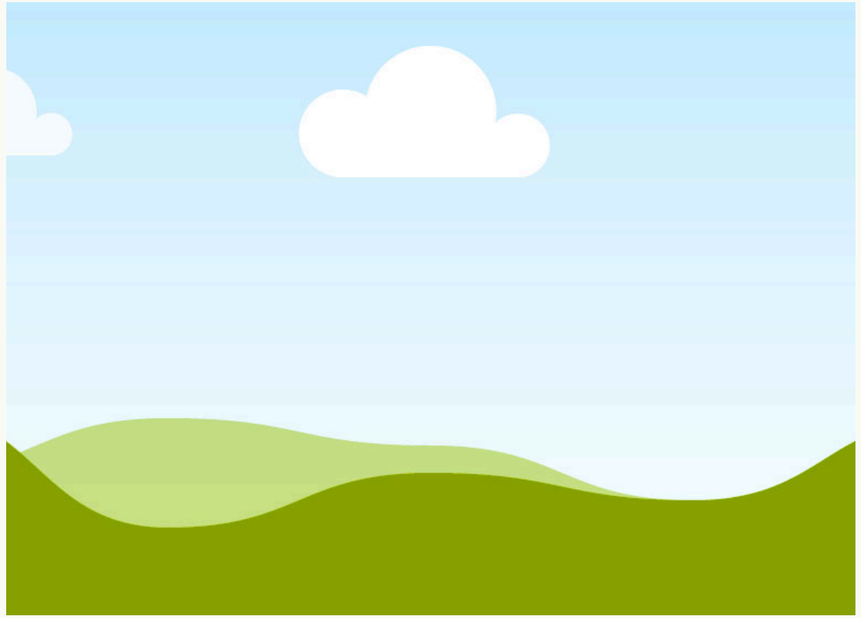
Visual: A graph showing a "Gap" between cost n and cost \alpha(n) \cdot n.



Algorithm 3.7 - Metric TSP (Factor 2)

- The Recipe:
  - Find an MST T of the graph.
  - Double every edge of the MST (makes it Eulerian).
  - Find an Euler tour T.
  - Short-cut the tour to visit each city once.

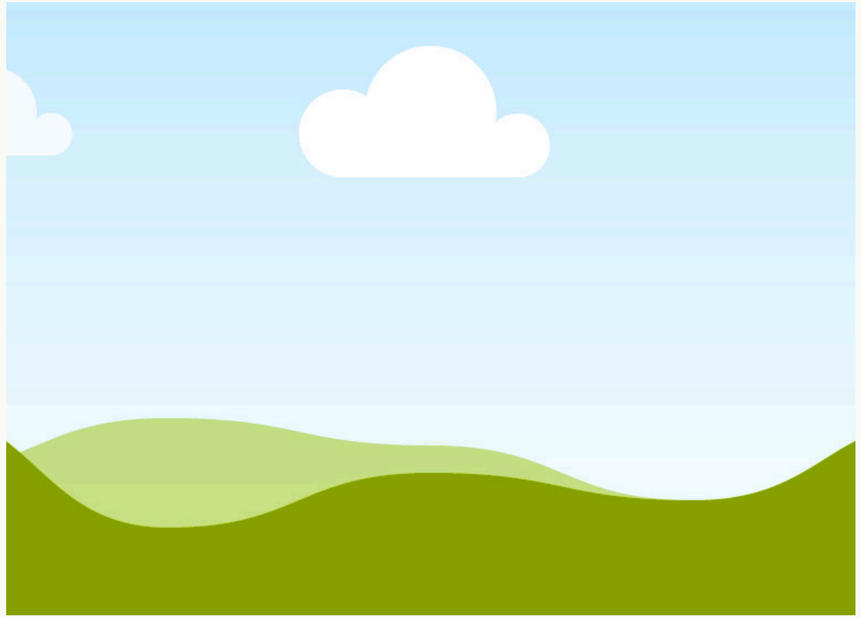
Visual: 1. MST \to 2. Doubled MST \to 3. Final Tour with shortcuts.



Theorem 3.8 - Factor 2 Guarantee

- The Statement: Algorithm 3.7 is a factor 2 approximation algorithm for metric TSP.
- The Meaning:
  - Our final tour cost will never be more than twice the optimal tour cost.
  - It is a simple, reliable way to get a "good enough" answer quickly.

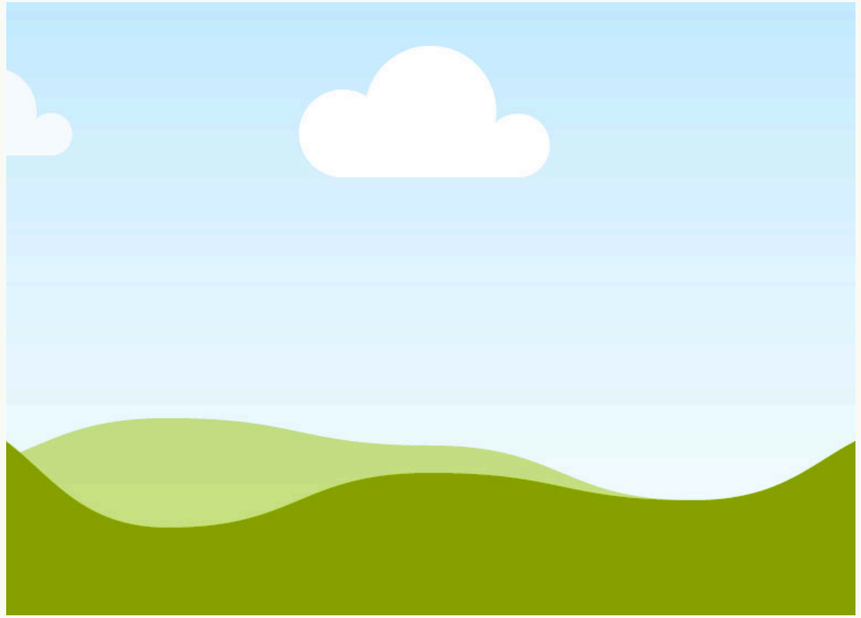
Visual: Equation: Cost(Tour) \leq 2 \cdot OPT.



Proof of Theorem 3.8

- Step-by-Step Logic:
  - Lower Bound: cost(MST) ≤ OPT (Removing one edge from the optimal tour gives a tree).
- Doubling: The Euler tour T uses each edge twice: cost(T) = 2 . cost(MST).
- Triangle Inequality: Short-cutting only reduces the cost: cost(Final) ≤ cost(T).
- Result: cost(Final) ≤ 2 . cost(MST) ≤ 2 . OPT.

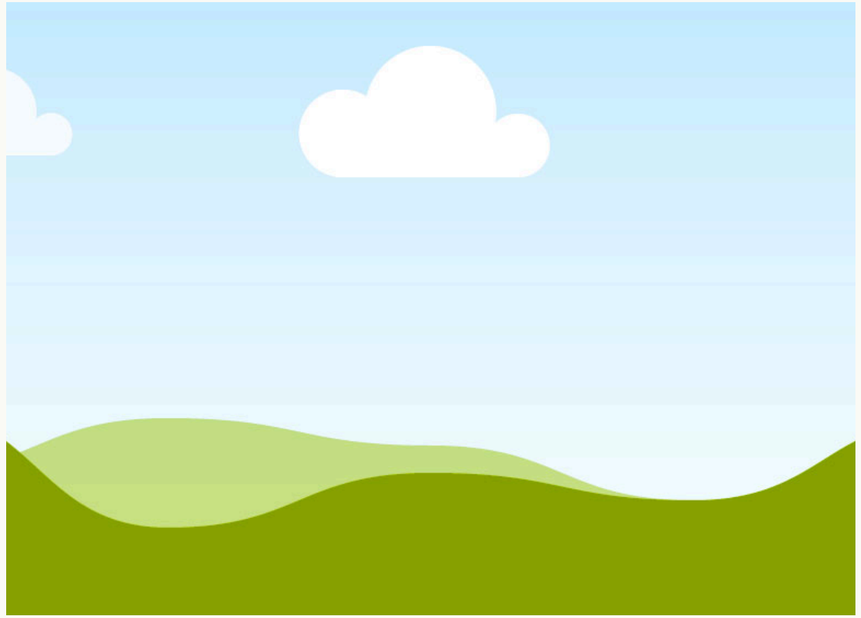
Visual: Mathematical derivation showing Cost \leq 2 \cdot OPT.



Example - Tight Case for Factor 2

- Scenario:
  - *n* vertices. Edges cost 1 or 2.
  - Optimal tour uses only "1" edges. Cost = n.
  - The algorithm picks a poor Euler tour order.
  - Final Cost = 2n - 2.
- Conclusion: This shows that factor 2 is the best guarantee we can give for this specific MST method.

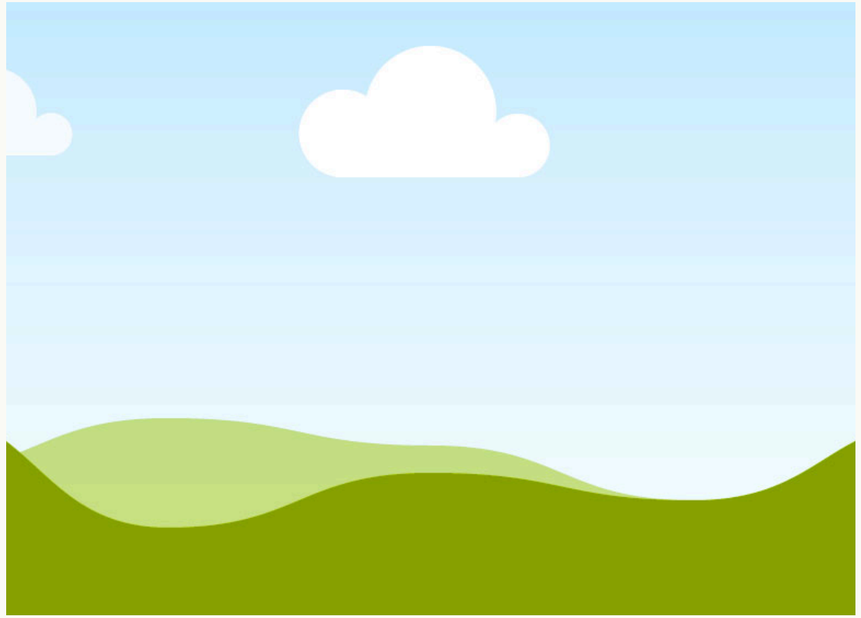
Visual: A graph for n=6 showing a star-shaped MST leading to a poor tour.



Improving the Factor - Why Double Everything?

- The Problem: Doubling every edge is "wasteful."
- The Insight:
  - We only need to make every vertex have an even degree to find an Euler tour.
  - In an MST, only some vertices have odd degrees.
  - We only need to fix those specific vertices.

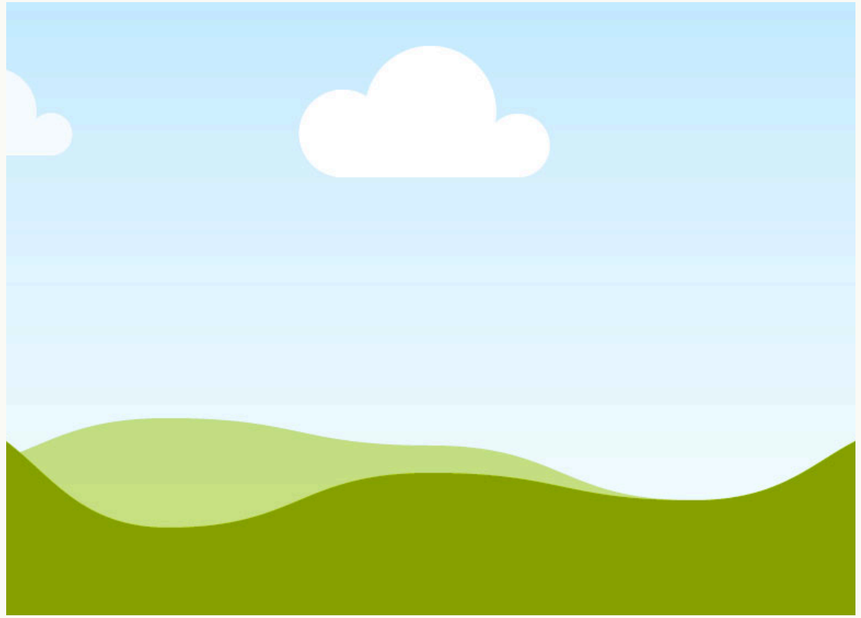
Visual: An MST with "Odd-degree" vertices highlighted in Red.



Algorithm 3.10 - Metric TSP (Factor 3/2)

- Steps (Christofides' Algorithm):
  - Find an MST T.
  - Find all odd-degree vertices V'.
  - Find a Minimum Cost Perfect Matching M on V'.
  - Combine $M + T$ to get an Eulerian graph.
  - Find Euler tour and short-cut.

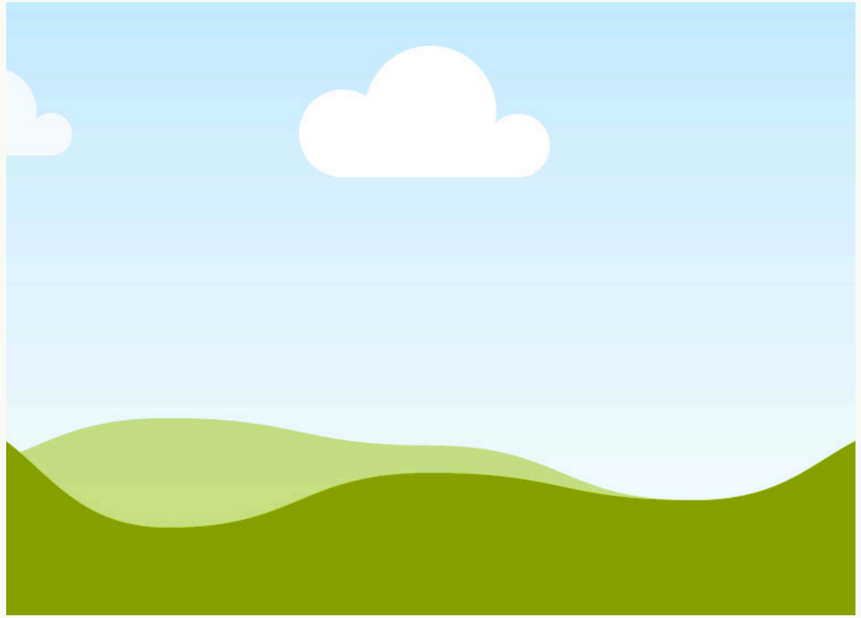
Visual: MST (lines) + Matching (dotted lines) \to Full Eulerian Graph.



Lemma 3.11 - Bound on Matching

- The Statement: Let V' be the set of odd-degree vertices. Let M be the minimum cost perfect matching on V'. Then cost(M) ≤ OPT/2.
- Key Idea: The matching is "cheap" because it only connects half the nodes from an optimal cycle.

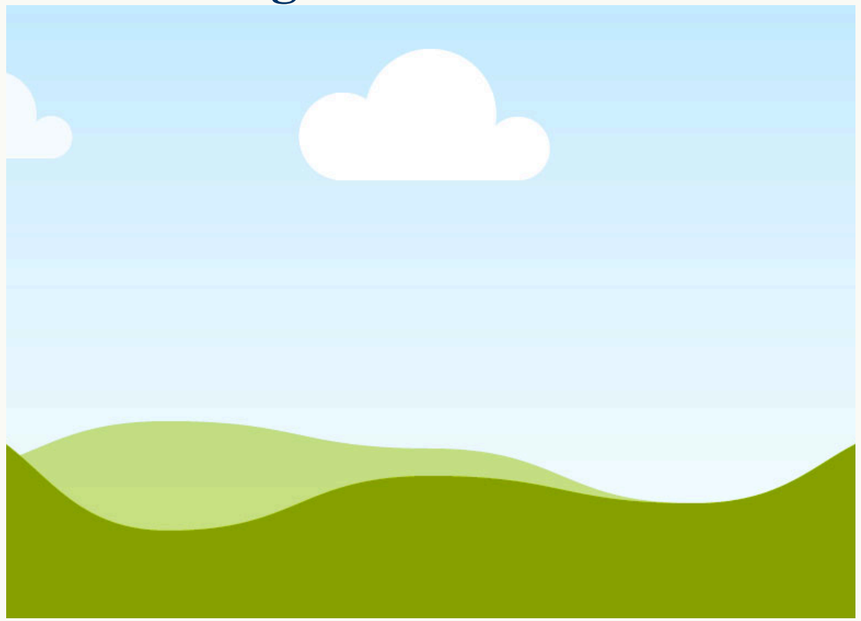
Visual: Equation: cost(M) (≤) 0.5 . OPT.



Proof of Lemma 3.11

- Logic:
  - Take the optimal tour OPT.
  - Short-cut it to visit only the odd vertices (V'). This is a cycle.
  - This cycle is a union of two perfect matchings (alternating edges).
  - By Triangle Inequality, cost(Cycle) ≤ OPT.
  - The cheaper of the two matchings must be ≤ cost(Cycle)/2 ≤ OPT/2.

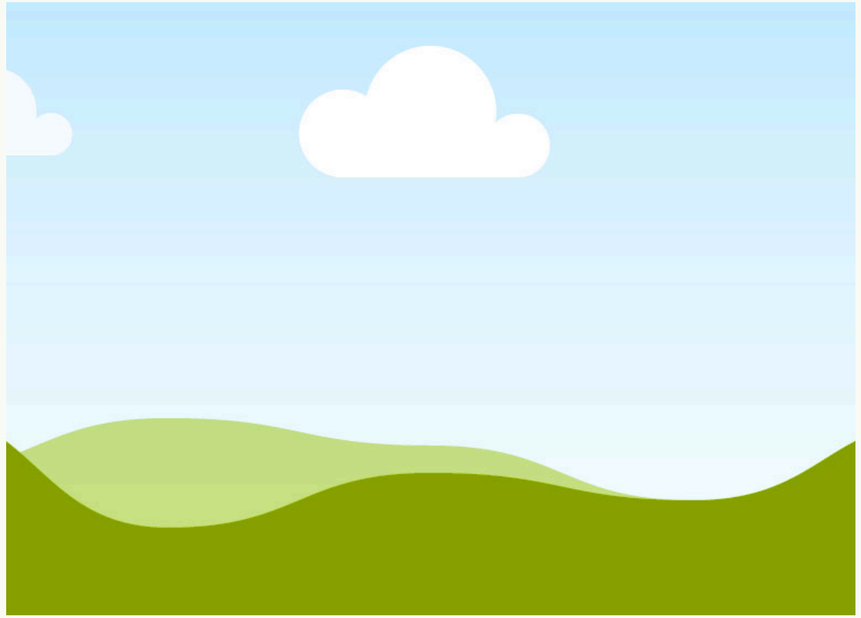
Visual: A cycle of 6 nodes with edges alternating colors (Red, Blue, Red, Blue...). Showing it splits into two separate matchings.



Theorem 3.12 - The 3/2 Factor

- The Analysis:
  - Final Cost = cost(MST) + cost(Matching)
  - cost(MST) ≤ OPT
  - cost(Matching) ≤ 0.5 . OPT
  - Total ≤ 1.5 . OPT
- Conclusion: Christofides' algorithm is a 3/2-approximation.

Visual: Bar chart comparing 1.0 (OPT) vs 1.5 (Algorithm) vs 2.0 (Standard MST).



Summary & Conclusion

- Key Takeaways:
  - Metric version is the "core" of both Steiner and TSP.
  - Steiner Tree: MST gives factor 2.
  - TSP: MST gives factor 2; Matching gives factor 1.5.

Questions?

Feel free to ask about the algorithms or proofs.

Thank You

For Your

Attention

Master in Digital Technology
TU Clausthal